

Module 09 Lab - Ensemble Methods

Objective: To learn how to use **ensemble methods**, which combine the predictions of several individual models to produce a final prediction that is more accurate and robust than any of the individual models alone. **In this lab, you will train and compare a single Decision Tree against a powerful ensemble method, the Random Forest.**

Part 1: The Wisdom of the Crowd

Concept: The core idea behind ensemble methods is the "wisdom of the crowd." If you ask one person a difficult question, they might be wrong. But if you ask a large, diverse group of people the same question and average their answers, the result is often surprisingly accurate. Ensemble methods apply this to machine learning. Instead of relying on a single model, we build a "committee" of models and have them vote on the final prediction. Two popular ensemble techniques:

- * **Bagging (Bootstrap Aggregating):** Trains multiple models (e.g., Decision Trees) on different random subsets of the training data. The **Random Forest** is the most famous example.*
- * **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by the previous one. Examples include Gradient Boosting and AdaBoost.

Part 2: Setup

We will use the Iris dataset to classify flower species.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare data
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random
```

Part 3: A Single Decision TreeA single Decision Tree is prone to overfitting. It can create very specific rules that match the

- ✓ training data perfectly but don't generalize well.**Your Task:** Train a single `DecisionTreeClassifier` and evaluate its accuracy.

```
# --- ENTER YOUR CODE HERE ---",
# 1. Create and train a single Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)
# 2. Make predictions and evaluate its accuracy",
y_pred_tree = single_tree.predict(X_test)
accuracy_tree = accuracy_score(y_test, y_pred_tree)
print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")
```

Accuracy of a single Decision Tree: 100.00%

Part 4: The Random Forest Ensemble**Concept:** A Random

Forest is an ensemble of many Decision Trees. It builds hundreds of trees, each trained on a random subset of the data and using a random subset of the features. To make a

- ✓ prediction, it gets a vote from every tree in the "forest" and chooses the class that gets the most votes.This process of averaging the votes of many diverse trees makes the Random Forest much more robust and less prone to overfitting than a single tree.**Your Task:** Train a `RandomForestClassifier` and compare its accuracy to the single tree.

```
# --- ENTER YOUR CODE HERE ---

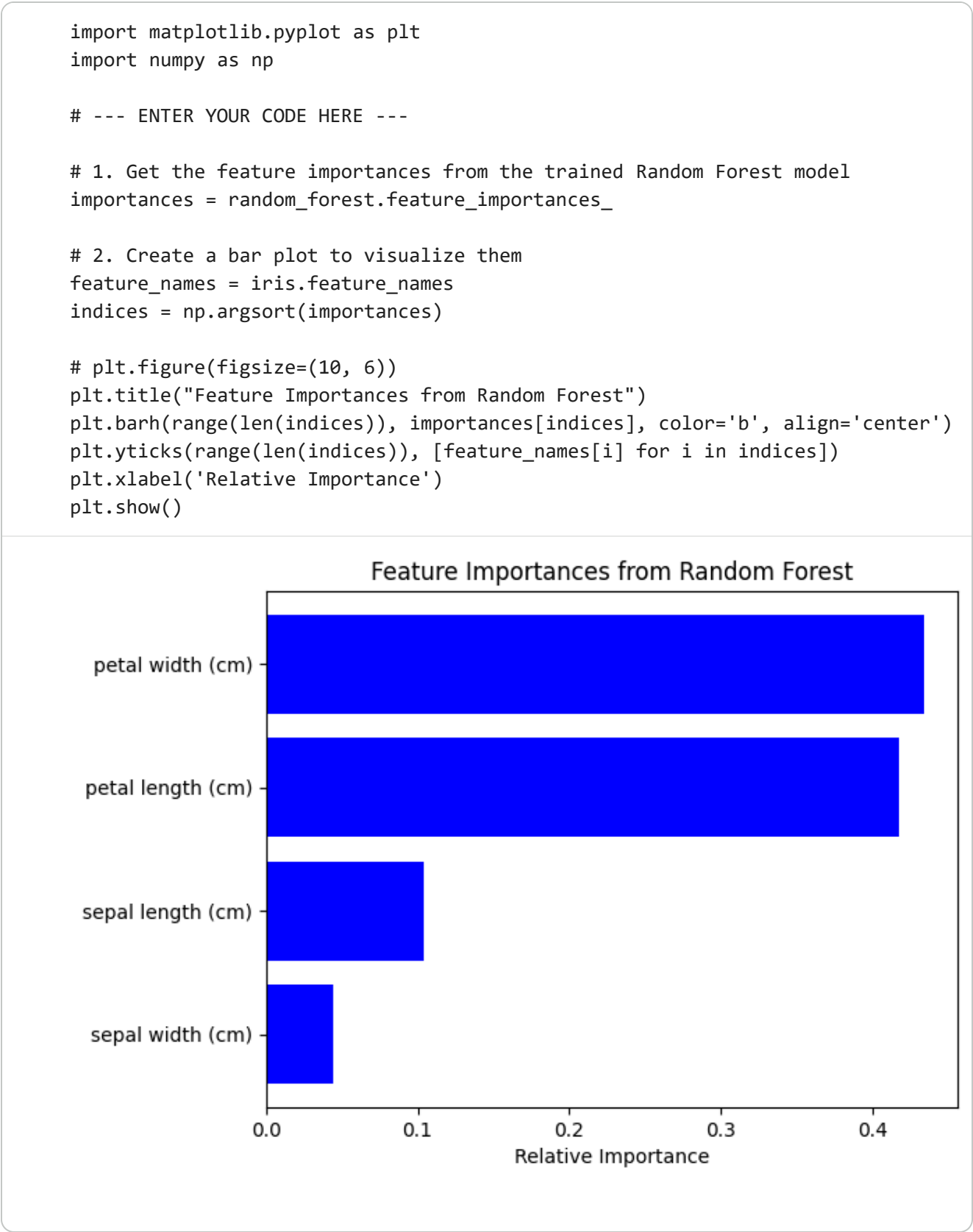
# 1. Create and train a Random Forest
# `n_estimators` is the number of trees in the forest.
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
random_forest.fit(X_train, y_train)

# 2. Make predictions and evaluate its accuracy
y_pred_rf = random_forest.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)

print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")
```

Accuracy of the Random Forest: 100.00%

Part 5: Feature Importance **Concept:** A great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions. **Your Task:** Extract and plot the feature importances from your trained Random Forest model.



 **Knowledge CheckInstructions:** Answer the following questions in this markdown cell.1. ****In your own words, what is the main idea behind ensemble methods?** The main idea behind ensemble methods is to make machine learning more accurate by using other machine Learning models to fix the

mistakes of the others. It's like doing a group assignment in sequential order, so if one person messes up, the next person can fix it. Why is a "committee" of models often better than a single expert model? A committee model can perform much better than an expert one; if a worse model messes up, there are other models that may see that and fix it, and using this with 2-3 models can cause almost no inaccuracies. While an expert alone may be better against one single model a expert model will still make some amount of mistakes, while less Accurate models can fix that by using each other

****2. ****Which model performed better in this lab, the single Decision Tree or the Random Forest? Is this what you expected? Why? Both models in this lab performed equally at 100% accuracy rating. This is not what I expected at all because throughout this course we have seen models in the 80% to 90% range, but I have never seen one as a 100% accurate model. I think this is because of the simplicity of the iris dataset and being easily distinguishable.

3. Looking at the feature importance plot, which two features were most important for classifying the iris flowers? Does this match what you might Expect from your own knowledge of flowers? The 2 most important features for classifying a flower according to the model are petal width and length. I would agree on the model because as most of the time flowers are defined by color shape and length of the petal when trying to figure out what type of flower it is.

Module 09 Lab - Ensemble Methods

- Objective:** To learn how to use **ensemble methods**, which combine the predictions of several individual models to produce a final prediction that is more accurate and robust than any of the individual models alone. **In this lab, you will train and compare a single Decision Tree against a powerful ensemble method, the Random Forest.**

Part 1: The Wisdom of the Crowd

Concept: The core idea behind ensemble methods is the "wisdom of the crowd." If you ask one person a difficult question, they might be wrong. But if you ask a large, diverse group of people the same question and average their answers, the result is often surprisingly accurate. Ensemble methods apply this to machine learning. Instead of relying on a single model, we build a "committee" of models and have them vote on the final prediction. Two popular ensemble techniques:

- * **Bagging (Bootstrap Aggregating):** Trains multiple models (e.g., Decision Trees) on different random subsets of the training data. The **Random Forest** is the most famous example.
- * **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by the previous one. Examples include Gradient Boosting and AdaBoost.

- Part 2: Setup We will use the Iris dataset to classify flower species.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare the data
iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Training samples: 105
Testing samples: 45

Part 3: A Single Decision Tree

- A single Decision Tree is prone to overfitting. It can create very specific rules that match the training data perfectly but don't generalize well. **Your Task:** Train a single `DecisionTreeClassifier` and evaluate its accuracy.

```
# Create and train a single Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)

# Make predictions and evaluate accuracy
y_pred_tree = single_tree.predict(X_test)
accuracy_tree = accuracy_score(y_test, y_pred_tree)

print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")
```

Accuracy of a single Decision Tree: 100.00%

Part 4: The Random Forest Ensemble
Concept: A Random Forest is an ensemble of many Decision Trees. It builds hundreds of trees, each trained on a random subset of the data and using a random subset of the features. To make a prediction, it gets a vote from every tree in the "forest" and chooses the class that gets the most votes. This process of averaging the votes of many diverse trees makes the Random Forest much more robust and less prone to overfitting than a single tree.
Your Task: Train a `RandomForestClassifier` and compare its accuracy to the single tree.

```
# Create and train a Random Forest
random_forest = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
random_forest.fit(X_train, y_train)

# Make predictions and evaluate accuracy
y_pred_rf = random_forest.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)

print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")
print(f"Accuracy difference: {accuracy_rf - accuracy_tree:.2%}")
```

Accuracy of the Random Forest: 100.00%
 Accuracy difference: 0.00%

Part 5: Feature Importance
Concept: A great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions.
Your Task: Extract and plot the feature importances from your trained Random Forest model.

```
# Get feature importances
importances = random_forest.feature_importances_
feature_names = iris.feature_names
indices = np.argsort(importances)

# Display values
importance_table = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values("Importance", ascending=False)

display(importance_table)

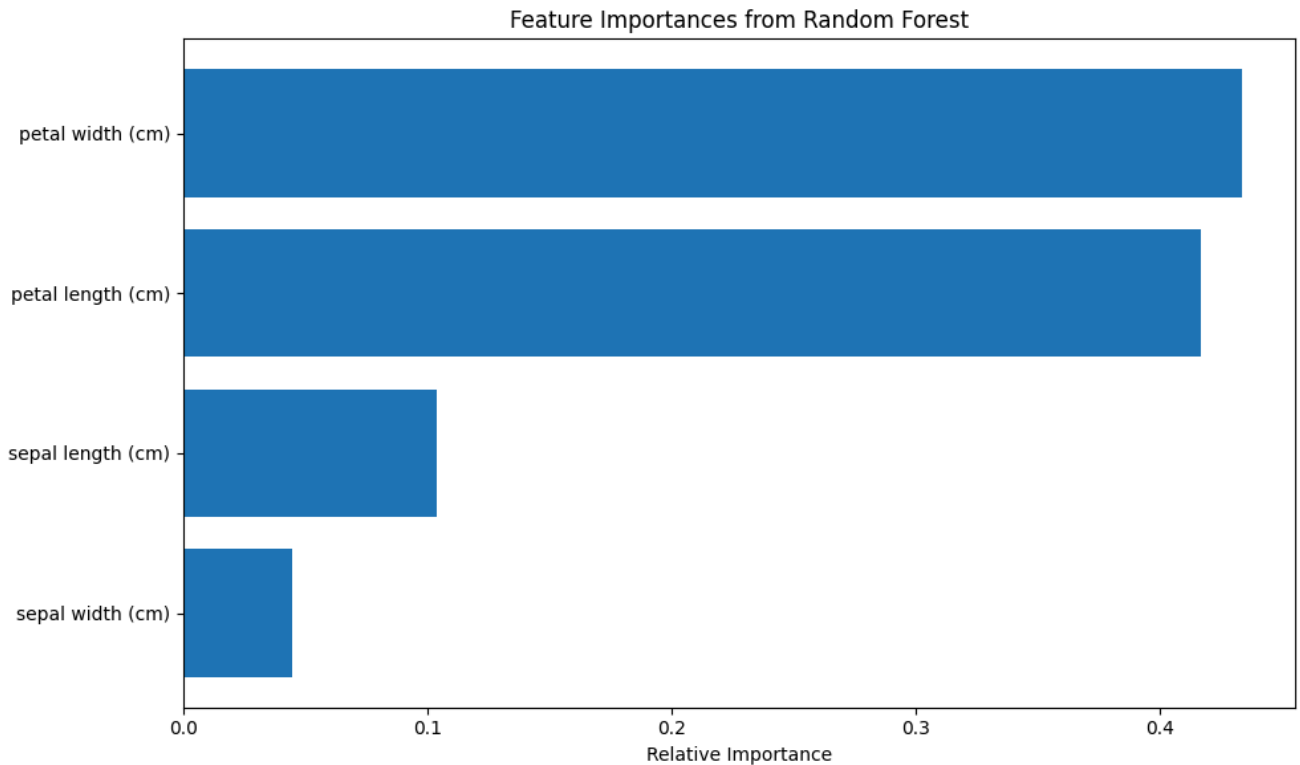
# Create a feature-importance plot
plt.figure(figsize=(10, 6))
plt.title("Feature Importances from Random Forest")
plt.barh(
    range(len(indices)),
    importances[indices],
    align="center"
)
plt.yticks(
```

```

    range(len(indices)),
    [feature_names[i] for i in indices]
)
plt.xlabel("Relative Importance")
plt.tight_layout()
plt.show()

```

	Feature	Importance	
3	petal width (cm)	0.433982	
2	petal length (cm)	0.417308	
0	sepal length (cm)	0.104105	
1	sepal width (cm)	0.044605	



Next steps: [Generate code with importance_table](#) [New interactive sheet](#)

Knowledge Check Instructions: Answer the following questions in this markdown cell.

- In your own words, what is the main idea behind ensemble methods? Why is a "committee" of models often better than a single expert model?** The main idea behind ensemble methods is to combine the predictions of several models instead of depending on only one model. A committee of models is often better because the individual models may make different mistakes. When their predictions are combined through voting or averaging, the errors of one model can be balanced by the correct predictions of others. This can produce a more accurate, stable, and reliable final result.
- Which model performed better in this lab, the single Decision Tree or the Random Forest? Is this what you expected? Why?** The single Decision Tree and the Random Forest both achieved 100% accuracy on the test data, so neither model performed better based on accuracy alone. I expected the Random Forest to perform at least as well as the Decision Tree because it combines 100 trees and is generally more resistant to overfitting. The Iris dataset is small and its flower classes are fairly separable, which may explain why both models classified every test example correctly.
- Looking at the feature importance plot, which two features were most important for classifying the iris flowers?** Does this match what you might expect from your own knowledge of flpetal width and petal length Petal width had an importance of about 43.4%, and petal length had an importance of about 41.7. This result makes sense because the size and shape of petals are major visible differences among iris species. Sepal measurements contributed less to the model's classifications.

